

Best practices to optimize data access performance from Amazon EMR and AWS Glue to Amazon S3

Customers are increasingly building data lakes to store data at massive scale in the cloud. It's common to use distributed computing engines, cloud-native databases, and data warehouses when you want to process and analyze your data in data lakes. [Amazon EMR](#) and [AWS Glue](#) are two key services you can use for such use cases. Amazon EMR is a managed big data framework that supports several different applications, including [Apache Spark](#), [Apache Hive](#), [Presto](#), [Trino](#), and [Apache HBase](#). AWS Glue Spark jobs run on top of Apache Spark, and distribute data processing workloads in parallel to perform extract, transform, and load (ETL) jobs to enrich, denormalize, mask, and tokenize data on a massive scale.

For data lake storage, customers typically use [Amazon Simple Storage Service](#) (Amazon S3) because it's secure, scalable, durable, and highly available. Amazon S3 is designed for 11 9's of durability and stores over 200 trillion objects for millions of applications around the world, making it the ideal storage destination for your data lake. Amazon S3 averages over 100 million operations per second, so your applications can easily achieve high request rates when using Amazon S3 as your data lake.

This post describes best practices to achieve the performance scaling you need when analyzing data in Amazon S3 using Amazon EMR and AWS Glue. We specifically focus on optimizing for Apache Spark on Amazon EMR and AWS Glue Spark jobs.

Optimizing Amazon S3 performance for large Amazon EMR and AWS Glue jobs

Amazon S3 is a very large distributed system, and you can scale to thousands of transactions per second in request performance when your applications read and write data to Amazon S3. Amazon S3 performance isn't defined per bucket, but per [prefix](#) in a bucket. Your applications can achieve at least 3,500 PUT/COPY/POST/DELETE or 5,500 GET/HEAD requests per second per prefix in a bucket. Additionally, there are no limits to the number of prefixes in a bucket, so you can horizontally scale your read or write performance using parallelization. For example, if you create 10 prefixes in an S3 bucket to parallelize reads, you could scale your read performance to 55,000 read requests per second. You can similarly scale writes by writing data across multiple prefixes.

You can scale performance by utilizing automatic scaling in Amazon S3 and scan millions of objects for queries run over petabytes of data. Amazon S3 automatically scales in response to sustained new request rates, dynamically optimizing performance. While Amazon S3 is internally optimizing for a new request rate, you receive HTTP 503 request responses temporarily until the optimization completes:

```
AmazonS3Exception: Please reduce your request rate. (Service: Amazon S3; Status Code: 503; Error Code: SlowDown)
```

Such situations require the application to retry momentarily, but after Amazon S3 internally optimizes performance for the new request rate, all requests are generally served without retries. One such situation is when multiple workers in distributed compute engines such as Amazon EMR and AWS Glue momentarily generate a high number of requests to access data under the same prefix.

When using Amazon EMR and AWS Glue to process data in Amazon S3, you can employ certain best practices to manage request traffic and avoid HTTP Slow Down errors. Let's look at some of these strategies.

Best practices to manage HTTP Slow Down responses

You can use the following approaches to take advantage of the horizontal scaling capability in Amazon S3 and improve the success rate of your requests when accessing Amazon S3 data using Amazon EMR and AWS Glue:

- Modify the retry strategy for Amazon S3 requests
- Adjust the number of Amazon S3 objects processed
- Adjust the number of concurrent Amazon S3 requests

We recommend choosing and applying the options that fit best for your use case to optimize data processing on Amazon S3. In the following sections, we describe best practices of each approach.

Modify the retry strategy for Amazon S3 requests

This is the easiest way to avoid HTTP 503 Slow Down responses and improve the success rate of your requests. To access Amazon S3 data, both Amazon EMR and AWS Glue use the [EMR File System](#) (EMRFS), which retries Amazon S3 requests with jitters when it receives 503 Slow Down responses. To improve the success rate of your Amazon S3 requests, you can adjust your retry strategy by configuring certain properties. In Amazon EMR, you can configure parameters in your `emrfs-site` configuration. In AWS Glue, you can configure the parameters in job parameters. You can adjust your retry strategy in the following ways:

- Increase the EMRFS default retry limit – By default, EMRFS uses an exponential backoff strategy to retry requests to Amazon S3. The default EMRFS retry limit is 15. However, you can increase this limit when you create a new cluster, on a running cluster, or at application runtime. To increase the retry limit, you can change the value of the `fs.s3.maxRetries` parameter. Note that you may experience longer job duration if you set a higher value for this parameter. We recommend experimenting with different values, such as 20 as a starting point, confirm the duration overhead of the jobs for each value, and then adjust this parameter based on your requirement.

- For Amazon EMR, use the AIMD retry strategy – With Amazon EMR versions 6.4.0 and later, EMRFS supports an alternative retry strategy based on an additive-increase/multiplicative-decrease (AIMD) model. This strategy can be useful in shaping the request rate from large clusters. Instead of treating each request in isolation, this mode keeps track of the rate of recent successful and throttled requests. Requests are limited to a rate determined from the rate of recent successful requests. This decreases the number of throttled requests, and therefore the number of attempts needed per request. To enable the AIMD retry strategy, you can set the `fs.s3.aimd.enabled` property to `true`. You can further refine the AIMD retry strategy using the [advanced AIMD retry settings](#).
- For Apache Iceberg tables, configure [table properties](#) to manage retries – Iceberg uses AWS SDK for Java version 2 (not EMRFS) to interact with S3. Following table properties are designed to manage retries for write commit operations:
 - `commit.retry.num-retries`
 - `commit.retry.min-wait-ms`
 - `commit.retry.max-wait-ms`
 - `commit.retry.total-timeout-ms`

Adjust the number of Amazon S3 objects processed

Another approach is to adjust the number of Amazon S3 objects processed so you have fewer requests made concurrently. When you lower the number of objects to be processed in a job, you use fewer Amazon S3 requests, thereby lowering the request rate or transactions per second (TPS) required for each job. Note the following considerations:

- Preprocess the data by aggregating multiple smaller files into fewer, larger chunks – For example, use [s3-dist-cp](#) or an [AWS Glue compaction blueprint](#) to merge a large number of small files (generally less than 64 MB) into a smaller number of optimally sized files (such as 128–512 MB). This approach reduces the number of requests required, while simultaneously improving the aggregate throughput to read and process data in Amazon S3. You may need to experiment to arrive at the optimal size for your workload, because creating extremely large files can reduce the parallelism of the job. For Apache Iceberg tables, [compact data files](#) using `rewriteDataFiles`. In case your Iceberg tables are managed in AWS Glue Data Catalog, use [automatic compaction](#).
- Use partition pruning to scan data under specific partitions – In Apache Hive and Hive Metastore-compatible applications such as Apache Spark or Presto, one table can have multiple partition folders. *Partition pruning* is a technique to scan only the required data in a specific partition folder of a table. It's useful when you want to read a specific portion from the entire table. To take advantage of predicate pushdown, you can use partition columns in the WHERE clause in Spark SQL or the filter expression in a DataFrame. In AWS Glue, you can also use a [partition pushdown predicate when creating DynamicFrames](#).
- For AWS Glue, enable job bookmarks – You can use [AWS Glue job bookmarks](#) to process continuously ingested data repeatedly. It only picks unprocessed data from

the previous job run, thereby reducing the number of objects read or retrieved from Amazon S3.

- For AWS Glue, enable bounded executions – [AWS Glue bounded execution](#) is a technique to only pick unprocessed data, with an upper bound on the dataset size or the number of files to be processed. This is another way to reduce the number of requests made to Amazon S3.

Adjust the number of concurrent Amazon S3 requests

To adjust the number of Amazon S3 requests to have fewer concurrent reads per prefix, you can configure Spark parameters. By default, Spark populates 10,000 tasks to list prefixes when creating Spark DataFrames. You may experience Slow Down responses, especially when you read from a table with highly nested prefix structures. In this case, it's a good idea to configure Spark to limit the number of maximum listing parallelism by decreasing the parameter `spark.sql.sources.parallelPartitionDiscovery.parallelism` (the default is `10000`).

To have fewer concurrent write requests per prefix, you can use the following techniques:

- Reduce the number of Spark RDD partitions before writes – You can do this by using `df.repartition(n)` or `df.coalesce(n)` in DataFrames. For Spark SQL, you can also use [query hints](#) like `REPARTITION` or `COALESCE`. You can [see the number of tasks \(=RDD partitions\) on the Spark UI](#).
- For AWS Glue, group the input data – If the datasets are made up of small files, we recommend [grouping the input data](#) because it reduces the number of RDD partitions, and reduces the number of Amazon S3 requests to write the files.
- Use the EMRFS S3-optimized committer – The [EMRFS S3-optimized committer](#) is used by default in Amazon EMR 5.19.0 and later, and AWS Glue 3.0. In AWS Glue 2.0, you can configure it in the job parameter `--enable-s3-parquet-optimized-committer`. The committer uses [Amazon S3 multipart uploads](#) instead of renaming files, and it usually reduces the number of HEAD/LIST requests significantly.

The following are other techniques to adjust the Amazon S3 request rate in Amazon EMR and AWS Glue. These options have the net effect of reducing parallelism of the Spark job, thereby reducing the probability of Amazon S3 Slow Down responses, although it can lead to longer job duration. We recommend testing and adjusting these values for your use case.

- Reduce the number of concurrent jobs – Start with the most read/write heavy jobs. If you configured cross-account access for Amazon S3, keep in mind that other accounts might also be submitting jobs to the prefix.
- Reduce the number of concurrent Spark tasks – You have several options:
 - For Amazon EMR, set the number of Spark executors (for example, the `spark-submit` option `--num-executors` and Spark parameter `spark.executor.instance`).
 - For AWS Glue, set the number of workers in the `NumberOfWorkers` parameter.

- For AWS Glue, change the `WorkerType` parameter to a smaller one (for example, `G.2X` to `G.1X`).
- Configure Spark parameters:
 - Decrease the number of `spark.default.parallelism`.
 - Decrease the number of `spark.sql.shuffle.partitions`.
 - Increase the number of `spark.task.cpus` (the default is `1`) to allocate more CPU cores per Spark task.

Apache Iceberg best practices to manage HTTP Slow Down responses

Some data lake applications that run on Amazon S3 handle millions or billions of objects and process petabytes of data. This can lead to prefixes that receive a high volume of traffic, which are typically detected through HTTP 503 (service unavailable) errors. To prevent this issue, use the following Iceberg properties:

- Set `write.distribution-mode` to `hash` or `range` so that Iceberg writes large files, which results in fewer Amazon S3 requests. This is effective for partitioned tables. This is the preferred configuration and should address the majority of cases.
- If you continue to experience 503 errors due to an immense volume of data in your workloads, you can set `write.object-storage.enabled` to `true` in Iceberg. This instructs Iceberg to hash object names and distribute the load across multiple, randomized Amazon S3 prefixes. Learn more in [Improve operational efficiencies of Apache Iceberg tables built on Amazon S3 data lakes](#).

For more information about these properties, see [Write properties](#) in the Iceberg documentation. For general best practices about Iceberg use cases on AWS, refer to [Using Apache Iceberg on AWS](#).

Conclusion

In this post, we described the best practices to optimize data access from Amazon EMR and AWS Glue to Amazon S3. With these best practices, you can easily run Amazon EMR and AWS Glue jobs by taking advantage of Amazon S3 horizontal scaling, and process data in a highly distributed way at a massive scale.

For further guidance, please reach out to [AWS Premium Support](#).

Appendix A: Configure CloudWatch request metrics

To monitor Amazon S3 requests, you can [enable request metrics](#) in [Amazon CloudWatch](#) for the bucket. Then, [define a filter](#) for the prefix. For a list of useful metrics to monitor, see [Monitoring metrics with Amazon CloudWatch](#). After you enable metrics, use the data in the metrics to determine which of the aforementioned options is best for your use case.

Appendix B: Configure Spark parameters

To configure Spark parameters in Amazon EMR, there are several options:

- spark-submit command – You can pass Spark parameters via the `--conf` option.
- Job script – You can set Spark parameters in the `SparkConf` object in the job script codes.
- Amazon EMR configurations – You can configure Spark parameters via API using Amazon EMR configurations. For more information, see [Configure Spark](#).

To configure Spark parameters in AWS Glue, you can configure AWS Glue job parameters using key `--conf` with value like `spark.hadoop.fs.s3.maxRetries=50`.

To set multiple configs, configure your job parameters using key `--conf` with value like `spark.hadoop.fs.s3.maxRetries=50 --conf spark.task.cpus=2`.